

MoF Phase 2: Per-Prompt-Set λ 架构分析

$\ell \in \mathbb{R}^{K \times T \times S}$ 的 Loss、梯度与 Reward 设计

Flow Factory

Abstract

基于 Phase 1 的分析结论，本文正式采用 per-prompt-set 扩展的 MoF 架构：logits $\ell \in \mathbb{R}^{K \times T \times S}$ ，其中 K = teacher 数量， T = 推理 timesteps， S = prompt set 数量。每个 prompt set 拥有独立的 teacher 混合策略。本文给出完整的 loss 推导、梯度公式、收敛性分析，并详细设计 per-set reward 策略——包括 reward normalization、advantage computation、OOD bonus 的精确公式和实现细节。

Contents

1	架构定义	2
1.1	符号表	2
1.2	Per-Set Velocity Field	2
1.3	参数空间几何	2
1.4	Inference 流程	3
2	Loss 函数	3
2.1	Per-Set DiffusionNFT Loss	3
2.2	Full Training Loss	3
3	梯度推导	4
3.1	Main Theorem	4
3.2	三重解耦性	4
3.3	Gradient 的分量分析	4
3.4	简化梯度 ($\beta = 1, w = 1, v^{\text{old}} \approx v^{\text{new}}$)	5
4	收敛性与最优解分析	5
4.1	Per-Set 最优解	5
4.2	Level 1 保证 ($\geq$ Single Teacher)	6
4.3	Level 2 条件 ($>$ Single Teacher)	6
4.4	温度 τ 与收敛形态	6
5	Reward 设计 (详细版)	6
5.1	Per-Set Composite Reward Definition	6
5.2	Advantage Computation (GDPO-Style Per-Reward Normalization)	7
5.3	OOD Bonus γ 的梯度效应	8
6	γ 的理论选择	8
6.1	γ 过大的风险	8
6.2	推荐 γ 范围	8

7	实现细节	8
7.1	Prompt Set Classification	8
7.2	Logits 初始化	9
7.3	EMA 与 Off-Policy	9
7.4	Training Batch 组织	9
7.5	Optimizer 配置	9
8	Reward 采样与评估	10
8.1	Sampling 阶段	10
8.2	Reward Evaluation 开销	10
9	理论对比与实验预期	10
9.1	对比总表	10
9.2	预期的 λ 收敛模式	11
10	总结	11
11	Reward-Advantage Pipeline: 理论推导与实现分析	12
11.1	Multi-Teacher / Multi-Reward / Multi-Dataset 对应关系	12
11.2	Reward Aggregation 公式 (旧方案 vs. 正确方案)	12
11.3	具体示例 (3-teacher config)	12
11.4	分布式训练下的正确性要求	13
11.5	实现中的潜在问题分析	13
11.6	推荐的实现改进	14
12	Batch-Level Normalization 时机分析	14
12.1	两种方案定义	14
12.2	数学等价性分析	15
12.3	对 NFT Loss 的影响	15
12.4	对比总结	16
12.5	推荐方案与理论依据	16
12.6	最终 Pipeline 伪代码	17
13	梯度消失分析与解决方案	17
13.1	问题现象	17
13.2	根因分析: 逐步推导	17
13.2.1	Step 1: 明确计算图	17
13.2.2	Step 2: Loss $\rightarrow$ Combined Velocity 的梯度 $\mathbf{g}$	18
13.2.3	Step 3: Combined Velocity $\rightarrow$ Weights 的梯度	18
13.2.4	Step 4: Weights $\rightarrow$ Logits 的梯度 (Softmax Jacobian —灾难发生处)	19
13.2.5	Step 5: 代入 LoRA 结构量化	20
13.3	为什么 <code>nft_beta</code> 不影响梯度	21
13.4	梯度消失的三重因素总结	21
13.5	核心困难的数学本质	21
13.6	对比: 为什么 LLM 的 Logits + Softmax 没有此问题	24
14	解决方案分析	25
14.1	Solution A: Direct Reward Loss (Contextual Bandit)	25
14.1.1	动机	25
14.1.2	Loss 定义	25
14.1.3	梯度分析	25
14.1.4	实际实现: Per-Teacher Advantage	26
14.2	Solution B: REINFORCE on Logit Space	26

14.2.1	Loss 定义	26
14.2.2	梯度分析	27
14.3	Solution C: Residual-Space NFT	27
14.3.1	动机	27
14.3.2	Loss 定义	27
14.3.3	梯度分析	27
14.4	Solution D: Unnormalized Additive Mixing	28
14.4.1	动机	28
14.4.2	Loss 定义	28
14.4.3	梯度分析	28
14.5	Solution 对比总表	29
14.6	推荐方案	30

1 架构定义

1.1 符号表

符号	含义
K	Teacher 数量 (= 3: GenEval, PickScore, OCR)
T	推理 timesteps 数量 (= num_inference_steps)
S	Prompt set 数量 (= 3: $\mathcal{P}_{\text{gen}}, \mathcal{P}_{\text{pick}}, \mathcal{P}_{\text{ocr}}$)
B	Batch size
d	Latent 维度 = $C \times H \times W$
$\ell \in \mathbb{R}^{K \times T \times S}$	可学习 logits (总参数量 = $K \times T \times S$)
τ	Softmax temperature
$\lambda_{k,i,s}$	Teacher k 在 timestep i 、prompt set s 上的权重
$v^{(k)}(x, t)$	Teacher k 的 frozen velocity field
$v_{\lambda_s}(x, t_i)$	Prompt set s 上的 combined velocity
$R_j(x)$	第 j 个 reward function (GenEval / PickScore / OCR)
$\mathcal{A}_s$	Prompt set s 上的 advantage
β	NFT beta (positive/negative interpolation)
γ	OOD bonus 系数

1.2 Per-Set Velocity Field

Definition 1 (Per-Set Combined Velocity). 对 prompt $p \in \mathcal{P}_s$ (属于 prompt set s)，定义：

$$v_{\lambda_s}(x, t_i) = \sum_{k=1}^K \lambda_{k,i,s} v^{(k)}(x, t_i), \quad \lambda_{k,i,s} = \frac{\exp(\ell_{k,i,s}/\tau)}{\sum_{k'=1}^K \exp(\ell_{k',i,s}/\tau)} \quad (1)$$

其中权重满足 $\lambda_{k,i,s} \geq 0$, $\sum_{k=1}^K \lambda_{k,i,s} = 1$, $\forall i, s$ 。

Remark 1 (与 Phase 1 Global λ 的关系). Global λ 是 per-set λ 的特例：当 $\ell_{k,i,s} = \ell_{k,i}$ 对所有 s 相同时，退化为 Phase 1 的 global 版本。Per-set 扩展引入了 S 倍的参数，但仍然极小 ($K \times T \times S = 3 \times 28 \times 3 = 252$ 参数)。

1.3 参数空间几何

Proposition 1 (搜索空间结构). Per-set MoF 的搜索空间为 S 个独立的乘积 simplex：

$$\Omega = \prod_{s=1}^S \prod_{i=1}^T \Delta^K = (\Delta^K)^{T \times S} \quad (2)$$

其中 $\Delta^K = \{\lambda \in \mathbb{R}_+^K : \sum_k \lambda_k = 1\}$ 是 K -simplex。

关键性质：不同 prompt set s 和 s' 的参数 $\ell_{:,i,s}$ 和 $\ell_{:,i,s'}$ 完全独立——梯度不交叉，优化完全解耦。

1.4 Inference 流程

Algorithm 1 Per-Set MoF Inference

Require: Prompt p , noise $x_T \sim \mathcal{N}(0, I)$, logits $\ell \in \mathbb{R}^{K \times T \times S}$

Ensure: Generated image x_0

```

1: Determine prompt set:  $s \leftarrow \text{classify}(p)$ 
2: Compute weights:  $\lambda_{:, :, s} \leftarrow \text{softmax}_k(\ell_{:, :, s} / \tau)$  ▷ shape  $(K, T)$ 
3:  $x \leftarrow x_T$ 
4: for  $i = 1, \dots, T$  do
5:   for  $k = 1, \dots, K$  do
6:      $v_i^{(k)} \leftarrow v^{(k)}(x, t_i)$  ▷ Frozen teacher forward
7:   end for
8:    $v_{\text{combined}} \leftarrow \sum_k \lambda_{k, i, s} \cdot v_i^{(k)}$  ▷ Weighted combination
9:    $x \leftarrow \text{scheduler\_step}(x, v_{\text{combined}}, t_i)$  ▷ ODE step
10: end for
11: return  $x$ 

```

2 Loss 函数

2.1 Per-Set DiffusionNFT Loss

对每个 prompt set s 独立定义 NFT loss。设 batch 中 prompt $p \in \mathcal{P}_s$:

Definition 2 (Per-Set NFT Loss (single timestep)).

$$\mathcal{L}_{i,s}(\ell_{:, :, s}) = \frac{1}{\beta} \left[r_s \cdot L_{i,s}^+ + (1 - r_s) \cdot L_{i,s}^- \right] \quad (3)$$

其中:

$$r_s = \frac{\mathcal{A}_s + A_{\max}}{2A_{\max}} \in [0, 1] \quad (4)$$

$$v_{i,s}^{\text{new}} = \sum_k \lambda_{k,i,s} v^{(k)}(x_{t_i}, t_i) \quad (\text{current policy for set } s) \quad (5)$$

$$v_{i,s}^{\text{old}} = \sum_k \bar{\lambda}_{k,i,s} v^{(k)}(x_{t_i}, t_i) \quad (\text{EMA policy for set } s) \quad (6)$$

$$v_{i,s}^+ = \beta v_{i,s}^{\text{new}} + (1 - \beta) v_{i,s}^{\text{old}} \quad (7)$$

$$v_{i,s}^- = (1 + \beta) v_{i,s}^{\text{old}} - \beta v_{i,s}^{\text{new}} \quad (8)$$

$$x_{0,s}^+ = x_{t_i} - \sigma_i v_{i,s}^+ \quad (9)$$

$$x_{0,s}^- = x_{t_i} - \sigma_i v_{i,s}^- \quad (10)$$

$$L_{i,s}^+ = \frac{\|x_{0,s}^+ - x_0\|^2}{(w_s^+)^2}, \quad L_{i,s}^- = \frac{\|x_{0,s}^- - x_0\|^2}{(w_s^-)^2} \quad (11)$$

w_s^+, w_s^- 为 detached mean absolute error (per-sample normalization)。

2.2 Full Training Loss

累积所有 timestep 和 batch samples:

$$\mathcal{L}_s(\ell_{:, :, s}) = \frac{1}{B_s \cdot T} \sum_{b \in \mathcal{B}_s} \sum_{i=1}^T \mathcal{L}_{i,s}^{(b)}(\ell_{:, :, s}) \quad (12)$$

其中 $\mathcal{B}_s$ 为当前 batch 中属于 prompt set s 的 sample 子集, $B_s = |\mathcal{B}_s|$ 。

Remark 2 (Per-Set 独立性). 总 loss 为各 set 的 loss 之和:

$$\mathcal{L}_{\text{total}}(\ell) = \sum_{s=1}^S \mathcal{L}_s(\ell_{:, :, s}) \quad (13)$$

由于 $\ell_{:, :, s}$ 只出现在 $\mathcal{L}_s$ 中, $\frac{\partial \mathcal{L}_{\text{total}}}{\partial \ell_{:, :, s}} = \frac{\partial \mathcal{L}_s}{\partial \ell_{:, :, s}}$ 。各 prompt set 的优化完全独立——如同并行训练 S 个独立的 global- λ MoF。

3 梯度推导

3.1 Main Theorem

Theorem 1 (Per-Set MoF Gradient). 对 prompt set s , single sample, single timestep i :

$$\frac{\partial \mathcal{L}_{i,s}}{\partial \ell_{k,i,s}} = \frac{2}{\tau} \lambda_{k,i,s} \left[v_i^{(k)} - v_{i,s}^{\text{new}} \right]^\top \underbrace{\left[\frac{r_s (x_{0,s}^+ - x_0)}{(w_s^+)^2} - \frac{(1-r_s) (x_{0,s}^- - x_0)}{(w_s^-)^2} \right]}_{\triangleq \mathbf{e}_{i,s} \in \mathbb{R}^d} \cdot (-\sigma_i) \quad (14)$$

且 $\frac{\partial \mathcal{L}_{i,s}}{\partial \ell_{k,j,s'}} = 0$ 当 $j \neq i$ 或 $s' \neq s$ 。

Proof. 推导与 Phase 1 Theorem 2.1 完全相同, 仅将 global $\lambda_{k,i}$ 替换为 per-set $\lambda_{k,i,s}$ 。各 set 的 sample 只对自己的 $\ell_{:, :, s}$ 产生梯度, 跨 set 梯度为零。□

3.2 三重解耦性

Corollary 1 (Triple Decoupling). Per-set MoF 的梯度具有三重解耦:

1. **Timestep 解耦:** $\frac{\partial \mathcal{L}_{i,s}}{\partial \ell_{k,j,s}} = 0$ for $j \neq i$
2. **Prompt-set 解耦:** $\frac{\partial \mathcal{L}_{i,s}}{\partial \ell_{k,i,s'}} = 0$ for $s' \neq s$
3. **Orthogonal teacher competition:** 在每个 (i, s) 固定的 slice 上, K 个 teacher 通过 softmax 竞争。增大 $\ell_{k,i,s}$ 必然减小 $\ell_{k',i,s}$ ($k' \neq k$) 的有效权重。

含义: 每个 (timestep, prompt set) pair 是一个完全独立的 K -way 选择问题。优化 252 个参数等价于并行优化 $T \times S = 84$ 个独立的 K -simplex 选择。

3.3 Gradient 的分量分析

将梯度 (14) 分解为三个语义明确的因子:

$$\frac{\partial \mathcal{L}_{i,s}}{\partial \ell_{k,i,s}} = \underbrace{\frac{2\sigma_i}{\tau} \lambda_{k,i,s}}_{\text{(I) Scale}} \cdot \underbrace{\left[v_i^{(k)} - v_{i,s}^{\text{new}} \right]}_{\text{(II) Direction}} \cdot \underbrace{(-\mathbf{e}_{i,s})}_{\text{(III) Reward signal}} \quad (15)$$

Factor (I): Scale.

- $\lambda_{k,i,s} \in [0, 1]$: 当前权重。权重越接近 0 或 1, 梯度越小 (softmax 饱和效应)。最大梯度在 $\lambda = 1/K$ 处 (均匀分布时, 各 teacher 梯度最灵敏)。
- σ_i : noise level at timestep i 。高噪声区域 (早期 timestep, $\sigma \approx 1$) 梯度较大; 低噪声区域 (晚期, $\sigma \rightarrow 0$) 梯度较小——这意味着早期 **timestep** 的权重学习更快。

Factor (II): Direction $[v_i^{(k)} - v_{i,s}^{\text{new}}]$.

- Teacher k 相对于当前 combined velocity 的“偏差方向”。
- 若 $\lambda_{k,i,s} = 1$ (已选中 teacher k)，则 $v_{i,s}^{\text{new}} = v_i^{(k)}$ ，此项为 0——已确定的选择不再变化。
- 若 $\lambda_{k,i,s} \ll 1$ ，则 $v_i^{(k)} - v_{i,s}^{\text{new}} \approx v_i^{(k)} - \sum_{k' \neq k} \lambda_{k'} v_i^{(k')}$ ——teacher k 与其他 teacher 混合的差异。

Factor (III): Reward signal $\mathbf{e}_{i,s}$.

- 由 advantage r_s 加权的 prediction error。
- 当 $r_s > 0.5$ (正 advantage, 当前 sample 好): $\mathbf{e}_{i,s}$ 主要由 L^+ 方向决定。
- 当 $r_s < 0.5$ (负 advantage, 当前 sample 差): $\mathbf{e}_{i,s}$ 主要由 $-L^-$ 方向决定。

3.4 简化梯度 ($\beta = 1, w = 1, v^{\text{old}} \approx v^{\text{new}}$)

在标准设定下 ($\beta = 1$, 忽略 normalization weight, small off-policy gap):

$$\frac{\partial \mathcal{L}_{i,s}}{\partial \ell_{k,i,s}} \approx \frac{2\sigma_i^2}{\tau} \lambda_{k,i,s} \left[v_i^{(k)} - v_{i,s}^{\text{new}} \right]^\top \left[(2r_s - 1)(v_{i,s}^{\text{new}} - v_i^*) \right] \quad (16)$$

其中 $v_i^* = (x_{t_i} - x_0)/\sigma_i$ 为 ground truth velocity。

梯度动力学解读

设 $\mathcal{A}_s > 0$ (好 sample, $r_s > 0.5$):

- 若 teacher k 的 velocity 比当前混合更接近 ground truth ($[v^{(k)} - v^{\text{new}}]^\top (v^{\text{new}} - v^*) < 0$)，则梯度为负 $\rightarrow$ gradient descent 增大 $\ell_{k,i,s} \rightarrow$ 增加 **teacher k** 在 **set s , timestep i** 上的权重。
- 若 teacher k 比混合更差，则权重减小。

设 $\mathcal{A}_s < 0$ (差 sample, $r_s < 0.5$):

- 反向信号: 惩罚当前混合方向, 探索远离当前配置的方向。

Per-set 的额外优势: set s 的梯度只看 set s 的 reward 信号——GenEval set 上的 reward 只影响 $\ell_{:, :, \text{gen}}$, OCR set 上的 reward 只影响 $\ell_{:, :, \text{ocr}}$ 。不同 set 的 reward 信号完全互不干扰。

4 收敛性与最优解分析

4.1 Per-Set 最优解

Theorem 2 (Per-Set 最优解的形式). 对 prompt set s , optimal logits $\ell_{:, :, s}^*$ 满足:

$$\ell_{:, :, s}^* = \arg \max_{\ell_{:, :, s}} \mathbb{E}_{p \in \mathcal{P}_s} [R_s(\text{ODE}(v_{\lambda_s}; x_T, p))] \quad (17)$$

其中 R_s 是 prompt set s 的 reward function (定义见 Section 5)。

4.2 Level 1 保证 ($\geq$ Single Teacher)

Theorem 3 (Guaranteed Non-Degradation). 对任意 prompt set s 和 teacher k :

$$\max_{\ell_{:,i},s} \mathbb{E}_{p \in \mathcal{P}_s} [R_s(v_{\lambda_s})] \geq R_s(v^{(k)}), \quad \forall k = 1, \dots, K \quad (18)$$

特别地, 取 $k = s$ (in-domain teacher):

$$\boxed{\mathbb{E}[R_s(v_{\lambda_s^*})] \geq \mathbb{E}[R_s(v^{(s)})]} \quad (\text{in-domain performance} \geq \text{single teacher}) \quad (19)$$

Proof. $\lambda_{k,i,s} = \delta_{k,s}$ (teacher s 恒取权重 1) 是可行解, 对应 $v_{\lambda_s} = v^{(s)}$ 。优化在包含此点的搜索空间上进行, 最优解不劣于此。对任意其他 teacher k 同理 ($\delta_{k,\cdot}$ 也是可行解)。□

Remark 3 (与 OPD 的对比). 这是 MoF per-set 架构相比 OPD-Average 的根本优势。OPD-Average 的最优解是 teacher 均值, 由 Jensen 不等式必然在 in-domain 上退化。Per-set MoF 则有构造性的下界保证。

4.3 Level 2 条件 ($>$ Single Teacher)

Theorem 4 (Strict Improvement Condition). Per-set MoF 严格超越 teacher s on set s 的充分条件为: 存在 timestep i^* 和 teacher $k^* \neq s$ 使得:

$$\mathbb{E}_{p \in \mathcal{P}_s} [R_s(\text{ODE}(v_{\lambda'}; x_T, p))] > \mathbb{E}_{p \in \mathcal{P}_s} [R_s(\text{ODE}(v^{(s)}; x_T, p))] \quad (20)$$

其中 λ' 仅在 timestep i^* 处与 $\delta_{k,s}$ 不同: $\lambda'_{k^*,i^*,s} = \epsilon > 0$, $\lambda'_{s,i^*,s} = 1 - \epsilon$ 。

直觉: 只需存在一个 timestep, 在该 timestep 上引入少量其他 teacher 的 velocity 能提升 set s 的 reward。

4.4 温度 τ 与收敛形态

Proposition 2 (温度对收敛的影响). 1. $\tau \rightarrow 0$ (低温极限): $\lambda_{k,i,s} \rightarrow \delta_{k, \arg \max_k \ell_{k,i,s}}$ 。每个 (i, s) 退化为 hard selection (winner-take-all)。搜索空间退化为 $\{1, \dots, K\}^{T \times S}$ ——离散组合优化。

2. $\tau \rightarrow \infty$ (高温极限): $\lambda_{k,i,s} \rightarrow 1/K$ 。所有 teacher 均匀混合, 等价于 teacher velocity 的算术平均。

3. τ 适中 (推荐 $\tau \in [0.5, 2.0]$): soft assignment, 允许梯度流动的同时保持一定的 specialization tendency。

Remark 4 (温度退火策略). 实践建议: 初始 $\tau = 1.0$ (充分探索), 训练后期逐步降低到 $\tau = 0.3-0.5$ (promote specialization)。这类似 Gumbel-Softmax 的退火策略。

5 Reward 设计 (详细版)

5.1 Per-Set Composite Reward Definition

Definition 3 (Per-Set Composite Reward with OOD Bonus). 对 prompt $p \in \mathcal{P}_s$ 生成的 image x , 设 $\mathcal{R}(x, s)$ 为该 sample 的 applicable reward 集合 (由 `applicable_sources` 过滤, 不适用的 reward 得到 NaN 被跳过):

$$\boxed{r_{\text{composite}}(x, s) = R_{\text{in}(s)}(x) + \gamma \cdot \frac{1}{|\mathcal{R}_{\text{ood}}|} \sum_{j \in \mathcal{R}_{\text{ood}}} R_j(x)} \quad (21)$$

其中 $\text{in}(s)$ 是 source s 的 in-domain reward (由 `TeacherConfig.reward_name` 显式指定), $\mathcal{R}_{\text{ood}} = \mathcal{R}(x, s) \setminus \{\text{in}(s)\}$, $\gamma \geq 0$ 是 OOD bonus 系数。

注意: 这里使用的是 **raw reward** (未做 normalization)。Reward scale 的差异通过下游 advantage 计算中的 group-std normalization 隐式消除。

Remark 5 (Reward 权重解读). • In-domain reward $R_{\text{in}(s)}$ 的有效系数为 1。

- 每个 OOD reward R_j ($j \neq \text{in}(s)$) 的有效系数为 $\frac{\gamma}{|\mathcal{R}_{\text{ood}}|}$ 。
- 当 $\gamma = 0$: 纯 in-domain 优化 (Level 1 保证最强, 但可能收敛到 teacher s 本身)。
- 当 $\gamma = 0.2$ (推荐): OOD 信号提供探索方向但不干扰主信号。

5.2 Advantage Computation (GDPO-Style Per-Reward Normalization)

MoF 采用 GDPO 的核心思想: 先对每个 **reward** 独立做 **group-level normalization** 得到 **per-reward advantage**, 然后在 **advantage** 空间做 **in-domain + OOD** 加权聚合。

Definition 4 (MoF Advantage Pipeline). 设 epoch 中有 N 个 sample (跨所有 rank gather 后), 按 prompt 分组为 $\mathcal{G}_1, \dots, \mathcal{G}_M$ (每个 group 有 K 个 sample)。

Step 1: Per-Reward Group Normalization. 对每个 reward k , 独立计算其 advantage:

$$a_k(x_i) = \frac{R_k(x_i) - \mu_{k,g}}{\max(\sigma_{k,g}, \epsilon)}, \quad \mu_{k,g} = \frac{1}{|\mathcal{G}_g|} \sum_{j \in \mathcal{G}_g} R_k(x_j), \quad \sigma_{k,g} = \text{std}(\{R_k(x_j)\}_{j \in \mathcal{G}_g}) \quad (22)$$

此步将各 reward 统一到 $\sim$ zero-mean, unit-variance 的 advantage 空间, 消除了 **reward scale** 差异 (如 GenEval $\in [0, 1]$ vs PickScore $\in [0.8, 0.95]$)。

Step 2: Per-Set Advantage Aggregation. 对 sample x_i 来自 source s :

$$\mathcal{A}_{\text{combined}}(x_i) = a_{\text{in}(s)}(x_i) + \gamma \cdot \frac{1}{|\mathcal{R}_{\text{ood}}|} \sum_{j \in \mathcal{R}_{\text{ood}}} a_j(x_i) \quad (23)$$

其中 $\mathcal{R}_{\text{ood}} = \mathcal{R}(x_i, s) \setminus \{\text{in}(s)\}$ 是适用于该 sample 的非 in-domain reward 集合。

Step 3: Batch-Level Normalization.

$$\mathcal{A}_{\text{final}}(x_i) = \frac{\mathcal{A}_{\text{combined}}(x_i)}{\max(\sigma_{\text{global}}, \epsilon)}, \quad \sigma_{\text{global}} = \text{std}(\{\mathcal{A}_{\text{combined}}(x_i)\}_{i=1}^N) \quad (24)$$

Remark 6 (与原始 GDPO 的关系). 原始 GDPO 的公式为:

$$\mathcal{A}_i^{\text{GDPO}} = \frac{\sum_k w_k \cdot a_k(x_i)}{\sigma_{\text{global}}} \quad (25)$$

其中 w_k 是全局静态权重。MoF 的区别在于: 权重是 **per-sample** 自适应的——对来自 source s 的 sample, in-domain reward 权重为 1, OOD reward 权重为 $\gamma/|\mathcal{R}_{\text{ood}}|$ 。这等价于 GDPO with sample-conditional weights。

Remark 7 (为什么在 Advantage 空间聚合而非 Raw Reward 空间). 如果在 raw reward 空间聚合 ($r = R_{\text{in}} + \gamma \cdot R_{\text{ood}}$), 然后做 group normalization, 会有以下问题:

- GenEval reward 范围 $[0, 1]$, PickScore 范围 $[0.8, 0.95]$ ——scale 不同导致 γ 的有效影响力不可预测。
- 同一 $\gamma = 0.2$ 对不同 reward pair 的实际效果差异巨大。

在 advantage 空间聚合 (各 reward 已归一化为 $\sim N(0, 1)$) 后:

- $\gamma = 0.2$ 意味着 OOD advantage 贡献 20% 的信号强度, 与 **reward 原始 scale** 无关。
- 不需要手动调整 reward weights 来补偿 scale 差异。

Remark 8 (Cross-Source Group 不混合的 Invariant). 关键约束: 同一 group 内的 K 个 sample 必须来自同一 source (因为共享同一 prompt)。因此 per-reward group normalization 中, 每个 group 内所有 sample 使用相同的 composite 公式——不会出现 group 内有的 sample 有某个 reward 而有的没有的情况。NaN (非 applicable) 只发生在跨 **group** 维度。

5.3 OOD Bonus γ 的梯度效应

Proposition 3 (OOD Bonus 如何引导探索). 设当前 $\lambda_{:,i,s}^* \approx \delta_{k,s}$ (已收敛到 in-domain teacher)。引入 $\gamma > 0$ 后, 梯度为:

$$\frac{\partial \mathcal{L}_{i,s}}{\partial \ell_{k',i,s}} \propto \lambda_{k',i,s} \left[v_i^{(k')} - v_i^{(s)} \right]^\top \mathbf{e}_{i,s} \quad (26)$$

其中 $\mathbf{e}_{i,s}$ 现在包含 OOD reward 的贡献。

当 OOD teacher k' 在某个 timestep i 上能提升 OOD reward $R_{k'}$ 而不显著损害 $R_{\text{in}(s)}$ 时:

- $\mathbf{e}_{i,s}$ 的 OOD 分量驱动 $\ell_{k',i,s}$ 增大。
- $\lambda_{k',i,s}$ 从 ~ 0 缓慢增长, 引入少量 teacher k' 的 velocity。
- 若这确实提升了综合 reward $r_{\text{composite}}$, 则正向 advantage 强化这一趋势。

本质: $\gamma > 0$ 为已收敛的 solution 提供了“突破 single-teacher 局部最优”的梯度推力。

6 γ 的理论选择

6.1 γ 过大的风险

Proposition 4 (γ Upper Bound). 若 γ 过大 (OOD reward 占主导), 则最优 $\lambda_{:,i,s}^*$ 将偏离 in-domain teacher, 导致 in-domain performance 退化。具体地, 当:

$$\gamma > \frac{\Delta R_{\text{in}}}{\Delta R_{\text{ood}}} \quad (27)$$

其中 ΔR_{in} = 从 teacher s 偏离时 R_s 的下降量, ΔR_{ood} = 同时 OOD reward 的提升量, 则 in-domain 会退化。

经验估计: 从实验数据看, OPD-Average (完全均匀混合) 导致 in-domain 下降 ~ 0.1 , OOD 提升 $\sim 0.01-0.03$ 。因此 $\Delta R_{\text{in}}/\Delta R_{\text{ood}} \sim 3-10$ 。安全的 γ 应远小于此。

6.2 推荐 γ 范围

推荐配置

策略	γ	预期效果
保守 (Level 1 为主)	0	确保 $\geq$ single teacher, 无 OOD bonus
推荐 (Level 1 + Level 2 探索)	0.2	in-domain 安全, OOD 有探索空间
激进 (Level 2 为主)	0.5	更强 OOD 探索, in-domain 可能微降

建议: 先用 $\gamma = 0$ 验证 Level 1 保证 (确认 MoF 至少达到 single teacher), 再切换 $\gamma = 0.2$ 探索 Level 2。

7 实现细节

7.1 Prompt Set Classification

Strategy 1 (Prompt Set Determination). 每个 prompt 携带 metadata 标注其 source set。具体来说, dataloader 中每个 batch element 包含 field `prompt_set_id` $\in \{0, 1, \dots, S-1\}$ 。

实现: 在 `MoFTrainer.optimize()` 中:

1. 从 batch 中提取 `prompt_set_id` tensor (shape $(B,)$)

2. 对每个 set s : $\mathcal{B}_s = \{b : \text{prompt_set_id}[b] = s\}$

3. 使用 $\ell_{:,i,s}$ 对 $\mathcal{B}_s$ 中的 sample 计算 loss 和梯度

7.2 Logits 初始化

Strategy 2 (Initialization for Per-Set Logits). 两种初始化策略:

Option A: Zeros (推荐用于验证 Level 1)

$$\ell_{k,i,s} = 0, \quad \forall k, i, s \quad (28)$$

初始权重为均匀 $\lambda_{k,i,s} = 1/K$ 。所有 teacher 起点平等。

Option B: Teacher-Biased (推荐用于 Level 2 探索)

$$\ell_{k,i,s} = \begin{cases} C & \text{if } k = s \\ 0 & \text{otherwise} \end{cases}, \quad C \approx 2.0 \quad (29)$$

初始权重偏向 in-domain teacher: $\lambda_{s,i,s} \approx 0.78$ ($K = 3, C = 2$), 其他 teacher ≈ 0.11 。从 single teacher 出发, 通过 reward-driven optimization 逐步引入其他 teacher 的贡献。

Remark 9 (Option B 的动机). Option B 的初始点即为 “near single-teacher” solution, 确保训练初期不退化。 $\gamma > 0$ 的 OOD bonus 从此基础上驱动探索。若 teacher $k' \neq s$ 在某些 timestep 上有正贡献, gradient 会逐步提升 $\ell_{k',i,s}$ 。

7.3 EMA 与 Off-Policy

Strategy 3 (Per-Set EMA). 对整个 logits tensor $\ell \in \mathbb{R}^{K \times T \times S}$ 维护 EMA:

$$\bar{\ell} \leftarrow (1 - \alpha_{\text{ema}}) \bar{\ell} + \alpha_{\text{ema}} \ell \quad (30)$$

其中 $\alpha_{\text{ema}} = 1 - \text{logits_ema_decay}$ (如 $\text{decay} = 0.99 \rightarrow \alpha = 0.01$)。

Sampling 使用 $\bar{\ell}$ (off-policy), training 使用 ℓ (on-policy), 产生 NFT 的 new/old policy gap。

7.4 Training Batch 组织

Strategy 4 (Mixed-Set Batch). 每个 training batch 包含来自所有 S 个 prompt set 的 sample (按比例混合):

$$B = B_{\text{gen}} + B_{\text{pick}} + B_{\text{ocr}}, \quad B_s \approx B/S \quad (31)$$

Batch 内并行: 一次 forward pass 产生所有 sample 的 teacher velocities, 然后按 `prompt_set_id` 分组计算各 set 的 loss 和梯度。

梯度累积: 由于三个 set 的梯度作用于 ℓ 的不同 slice ($\ell_{:,i,0}, \ell_{:,i,1}, \ell_{:,i,2}$), 可以直接 in-place 累积, 无冲突。

7.5 Optimizer 配置

Strategy 5 (Per-Set Optimizer). 虽然参数解耦, 但实践中使用单个 **AdamW optimizer** 管理整个 $\ell \in \mathbb{R}^{K \times T \times S}$ (因为 Adam 的 momentum/variance 是 per-parameter 的, 天然支持不同 slice 的不同更新频率)。

- **Learning rate:** 推荐 `logits_lr` $\in [10^{-2}, 10^{-1}]$ (logits 空间需要较大 lr)。
- **Weight decay:** 0 (logits 不需要正则化, softmax 自带 bounded output)。
- **Gradient clipping:** `max_grad_norm` = 1.0 (防止单个高 advantage sample 导致 logits 跳变过大)。

8 Reward 采样与评估

8.1 Sampling 阶段

Algorithm 2 Per-Set MoF Sampling

Require: EMA logits $\bar{\ell}$, dataloader (mixed prompt sets)

Ensure: Samples with rewards and prompt set labels

```

1: for each batch from dataloader do
2:   for each sample  $b$  in batch do
3:      $s_b \leftarrow \text{prompt\_set\_id}[b]$ 
4:     Generate  $x_b$  using  $\bar{\ell}_{[:, :, s_b]}$  (Algorithm 1)
5:   end for
6:   Compute rewards:  $R_{\text{gen}}(x_b)$ ,  $R_{\text{pick}}(x_b)$ ,  $R_{\text{ocr}}(x_b)$  for all  $b$ 
7:   Compute per-set reward:  $R_{s_b}(x_b)$  using eq. (21)
8: end for
  
```

8.2 Reward Evaluation 开销

Remark 10 (三倍 Reward 计算). 每个 sample 需要计算所有三个 reward function (不仅是自己 set 的), 因为 OOD bonus 需要跨 set reward. 对于仅 inference-based rewards (如 PickScore, GenEval), 这是轻量级的 forward pass. OCR reward 需要 OCR model inference. 总 reward 评估开销 $\approx 3 \times$ 单 reward 成本。

9 理论对比与实验预期

9.1 对比总表

	OPD-Avg	OPD-Route	MoF (global)	MoF (per-set)
Parameters	$\sim 10^7$	$\sim 10^7$	$K \times T$	$K \times T \times S$
In-domain 保证	$\times < \text{single}$	$\approx \text{single}$	弱 (折衷)	$\checkmark \geq \text{single}$
Level 2 可能	$\times$	$\times$	中 (受折衷限制)	$\checkmark$ 高
Cross-set 干扰	强 (共享 θ)	无 (分离)	有 (共享 λ)	无
Reward needed	$\times$	$\times$	$\checkmark$	$\checkmark$
Optimal obj.	KL min	KL min	Reward max	Reward max

9.2 预期的 λ 收敛模式

预期实验结果

训练收敛后，我们预期看到以下 λ pattern:

Set = GenEval ($\ell_{:, :, \text{gen}}$):

- 早期 timestep (t_1-t_8): GenEval teacher 占主导 ($\lambda_{\text{gen}} \sim 0.8-0.9$) , 因为 layout/composition 在早期确定。
- 晚期 timestep ($t_{20}-t_{28}$): 可能少量 PickScore teacher 贡献 ($\lambda_{\text{pick}} \sim 0.1-0.2$) , 因为清晰的 rendering 有助于 GenEval 的属性判断。

Set = OCR ($\ell_{:, :, \text{ocr}}$):

- 早期 timestep: 可能 GenEval teacher 有贡献 (文字 spatial positioning)。
- 中期 timestep (t_8-t_{18}): OCR teacher 占主导 (笔画形成阶段)。
- 晚期 timestep: OCR teacher 仍主导, 可能少量 PickScore 提升非文字区域美感。

Set = PickScore ($\ell_{:, :, \text{pick}}$):

- 全程 PickScore teacher 占主导 ($\lambda_{\text{pick}} \sim 0.7-0.9$) , 因为 aesthetics 是全局属性。
- 早期 timestep 可能少量 GenEval teacher 贡献 (正确的 composition 本身提升美感)。

10 总结

Phase 2 核心设计总结

1. **架构**: $\ell \in \mathbb{R}^{K \times T \times S}$ (252 个参数), 每个 prompt set 独立的 teacher 混合策略。
2. **Loss**: Per-set DiffusionNFT loss, 各 set 完全解耦。梯度具有三重正交性 (timestep $\times$ prompt-set $\times$ teacher-softmax)。
3. **Reward**: $r_{\text{composite}}(x, s) = R_{\text{in}(s)}(x) + \gamma \cdot \text{mean}(R_{\text{ood}})$, normalization 交给 GDPO-style advantage。
4. **Initialization**: Teacher-biased ($C = 2$) 从 near-single-teacher 出发, 保证初始不退化。
5. **保证**: Level 1 ($\geq$ single teacher) 有构造性证明。Level 2 ($>$ single teacher) 由 OOD bonus 驱动探索。
6. **Expected outcome**: 每个 prompt set 上至少匹配 single teacher in-domain performance, 同时在 timestep 维度上发现 cross-teacher 互补性, 实现 OOD 提升甚至 in-domain 超越。

11 Reward-Advantage Pipeline: 理论推导与实现分析

11.1 Multi-Teacher / Multi-Reward / Multi-Dataset 对应关系

Source (dataset)	In-domain Teacher	In-domain Reward	Applicable Rewards
geneval	GenEval Teacher	geneval	geneval, pick_score
pickscore	PickScore Teacher	pick_score	pick_score
ocr	OCR Teacher	ocr	ocr, pick_score

Definition 5 (对应关系的三层结构). 1. **Teacher** $\rightarrow$ **Source**: 由 `TeacherConfig.sources` 指定。Teacher k 的 velocity 用于所有 source 的 sample (MoF 中所有 teacher 都 forward), 但 `sources` 决定了哪个 teacher 是该 source 的 “in-domain teacher” (用于 `teacher_biased` 初始化)。

2. **Source** $\rightarrow$ **In-domain Reward**: 由 `TeacherConfig.reward_name` 显式指定。决定了对 source s 的 sample 计算 composite reward 时, 哪个 reward 是主信号 (权重 = 1)。

3. **Reward** $\rightarrow$ **Applicable Sources**: 由 `RewardConfig.applicable_sources` 指定。决定了哪些 source 的 sample 可以被该 reward model 评分 (不适用的 sample 得到 NaN)。

11.2 Reward Aggregation 公式 (旧方案 vs. 正确方案)

旧方案 (raw reward 空间聚合 $\rightarrow$ 再 normalize):

$$r_{\text{composite}}(x, s) = R_{\text{in}(s)}(x) + \gamma \cdot \text{mean}(R_{\text{ood}}) \xrightarrow{\text{group norm}} \mathcal{A}_i = \frac{r_i - \mu_g}{\sigma_g \cdot \sigma_{\text{global}}} \quad (\text{旧})$$

问题: GenEval $\in [0, 1]$ 与 PickScore $\in [0.8, 0.95]$ 的 scale 差异使 γ 的有效强度不可控。

正确方案 (per-reward normalize $\rightarrow$ advantage 空间聚合): 与 Section 4.2 的 Definition 4 一致:

$$\text{Step 1: } a_k(x_i) = \frac{R_k(x_i) - \mu_{k,g}}{\sigma_{k,g}} \quad (\text{per-reward, per-group}) \quad (\text{eq. 22})$$

$$\text{Step 2: } \mathcal{A}_{\text{combined}}(x_i) = a_{\text{in}(s)}(x_i) + \gamma \cdot \text{mean}(a_{\text{ood}}) \quad (\text{eq. 23})$$

$$\text{Step 3: } \mathcal{A}_{\text{final}}(x_i) = \mathcal{A}_{\text{combined}}(x_i) / \sigma_{\text{global}} \quad (\text{eq. 24})$$

此方案中 $\gamma = 0.2$ 的语义清晰: OOD advantage (已归一化为 $\sim N(0, 1)$) 贡献 20% 的信号强度, 与 reward 原始 scale 完全无关。

11.3 具体示例 (3-teacher config)

- geneval sample: $\mathcal{A} = a_{\text{geneval}} + 0.2 \cdot a_{\text{pick_score}}$
- pickscore sample: $\mathcal{A} = a_{\text{pick_score}}$ (无 OOD applicable)
- ocr sample: $\mathcal{A} = a_{\text{ocr}} + 0.2 \cdot a_{\text{pick_score}}$

其中每个 a_k 已经是 group-normalized (zero-mean, unit-variance within group)。

11.4 分布式训练下的正确性要求

关键约束

在 `distributed_k_repeat` sampler 下, 同一 group 的 K 个 sample 可能分散在不同 rank 上。正确的 group-mean 需要跨 **rank gather**——只用本地 samples 计算 group-mean 会得到错误结果。

当前实现通过 `AdvantageProcessor.collect_group_rewards()` 解决:

1. Pack rewards + unique_id $\rightarrow (B, 2)$ tensor
2. `accelerator.gather()` $\rightarrow (W*B, 2)$ global tensor
3. 在 global tensor 上计算 group-mean / global-std
4. `_to_local()` scatter 回各 rank

11.5 实现中的潜在问题分析

1. Cross-source Group 混合问题 (中等风险)

当前 sampling 使用 `_interleaved_source_iter()` 交替产出不同 source 的 batch。每个 batch 对应一个 source, batch 内的 samples 共享相同的 `unique_id` (同 prompt 的 K 个 sample 在同一 batch 内)。

但: 如果 `group_size=16` 且 `per_device_batch_size=8`, 则一个 group 需要 2 个 batch 来生成 16 个 sample (`batch size  $\times$  8 GPU = 64 per batch`, 但 `unique_sample_num_per_epoch` 控制不同 prompt 的数量)。

关键问题: 同一 group 内的所有 sample 必须来自同一 source (因为它们共享同一个 prompt)。当前的 `_interleaved_source_iter()` 保证每个 batch 是单 source 的, 但同一 prompt 的 K 个重复 sample 可能跨多个 batch。由于 interleaved 模式 round-robin 切换 source, 这些重复 sample 确实在同一个 source 的 batch 中生成——正确, 无问题。

2. Group-Std vs. Global-Std 的语义差异 (低风险但需注意)

当前实现做了两层 normalization: 先 group-std (eq. 22), 再 global-std (eq. 24)。这等价于:

$$\mathcal{A}_i = \frac{r_i - \mu_g}{\sigma_g \cdot \sigma_{\text{global}}} \quad (32)$$

如果不同 source 的 group 内 reward variance 差异很大 (如 `geneval` 的 `group-std` ≈ 0.3 , `pickscore` 的 `group-std` ≈ 0.02), `group-std` normalization 后它们的 scale 已经对齐。`global-std` 此时主要作用是稳定整体 scale (不改变相对大小)。

替代方案: 只用 group-mean 减去 (不除 group-std), 然后用 global-std 统一 normalize。这保留了不同 source 的 reward 信号强度差异, 可能更好:

$$\mathcal{A}_i^{\text{alt}} = \frac{r_i - \mu_g}{\sigma_{\text{global}}} \quad (33)$$

但当前的双层 normalization 在 GDPO 论文中已验证有效, 暂不修改。

3. Composite Reward 的 Scale 不一致 (需注意)

由于 `applicable_sources` 过滤, 不同 source 的 sample 的 composite reward 组成不同:

- `geneval` sample: $R_{\text{geneval}} + 0.2 \cdot R_{\text{pick}}$ (两项之和)
- `pickscore` sample: R_{pick} (单项)

两者的原始 scale 不同。但由于 group-mean 是组内计算（同一 prompt $\Rightarrow$ 同一 source $\Rightarrow$ 同一 composite 公式），且 group-std 归一化后 scale 对齐，这不是问题。

4. `_to_local()` 返回类型（minor bug）

`AdvantageProcessor._to_local()` 在 `group_on_same_rank=True` 时返回 `torch.Tensor`（可能在 GPU 上），在 `group_on_same_rank=False` 时返回 CPU numpy array 经 `torch.as_tensor` 转换。后续 `float(adv)` 对 GPU tensor 有隐式 D2H transfer。建议统一 `.cpu().numpy()` 后再 store。

11.6 推荐的实现改进

短期可执行的改进

1. **fix:** 在 `prepare_feedback` 中 `local_advantages` 转为 numpy 后再逐 sample 存储（避免逐个 GPU $\rightarrow$ CPU transfer）。
2. **enhancement:** 为 pickscore source 也配置 OOD reward（如 `geneval`），使其也能受益于 cross-teacher signal:

```
- name: "geneval"
  applicable_sources: [geneval, pickscore]
```

这样 pickscore sample 的 `composite = Rpick + 0.2 · Rgeneval`，获得更丰富的梯度信号。

3. **monitoring:** 添加 per-source advantage mean 的 logging，确认各 source 的 advantage 分布是否平衡（不应有某个 source 的 advantage 系统性偏高或偏低）。

12 Batch-Level Normalization 时机分析

核心问题：在 MoF 的 3-step advantage pipeline 中，batch-level normalization (σ_{global} 除法) 应放在 **per-reward advantage** 聚合之前还是之后？

12.1 两种方案定义

Definition 6 (Option A: Batch-Level Normalization **Before** Aggregation).

$$\text{Step 1: } a_k(x_i) = \frac{R_k(x_i) - \mu_{k,g}}{\max(\sigma_{k,g}, \epsilon)} \quad (\text{per-reward, per-group}) \quad (34)$$

$$\text{Step 1.5: } \hat{a}_k(x_i) = \frac{a_k(x_i)}{\max(\sigma_{k,\text{batch}}, \epsilon)}, \quad \sigma_{k,\text{batch}} = \text{std}(\{a_k(x_j)\}_{j=1}^N) \quad (35)$$

$$\text{Step 2: } \mathcal{A}_{\text{final}}(x_i) = \hat{a}_{\text{in}(s)}(x_i) + \gamma \cdot \frac{1}{|\mathcal{R}_{\text{ood}}|} \sum_{j \in \mathcal{R}_{\text{ood}}} \hat{a}_j(x_i) \quad (36)$$

特点：每个 reward 的 advantage 被独立归一化为 batch-level unit variance，然后直接聚合。聚合后不再做额外 **normalization**。

Definition 7 (Option B: Batch-Level Normalization **After** Aggregation).

$$\text{Step 1: } a_k(x_i) = \frac{R_k(x_i) - \mu_{k,g}}{\max(\sigma_{k,g}, \epsilon)} \quad (\text{per-reward, per-group}) \quad (37)$$

$$\text{Step 2: } \mathcal{A}_{\text{combined}}(x_i) = a_{\text{in}(s)}(x_i) + \gamma \cdot \frac{1}{|\mathcal{R}_{\text{ood}}|} \sum_{j \in \mathcal{R}_{\text{ood}}} a_j(x_i) \quad (38)$$

$$\text{Step 3: } \mathcal{A}_{\text{final}}(x_i) = \frac{\mathcal{A}_{\text{combined}}(x_i)}{\max(\sigma_{\text{global}}, \epsilon)}, \quad \sigma_{\text{global}} = \text{std}(\{\mathcal{A}_{\text{combined}}(x_j)\}_{j=1}^N) \quad (39)$$

特点：先在 advantage 空间做 in-domain + OOD 聚合，然后对聚合后的 combined advantage 做统一的 batch-level normalization。

12.2 数学等价性分析

Proposition 5 (Group-Std 后 Batch-Std 近似为恒等). 设 Step 1 的 group normalization 正确工作（即各 group 内 sample 数量 ≥ 4 ），则：

$$\sigma_{k,\text{batch}} = \text{std} \left(\left\{ \frac{R_k(x_j) - \mu_{k,g_j}}{\sigma_{k,g_j}} \right\}_{j=1}^N \right) \approx 1 \quad (40)$$

原因：各 group 独立贡献 zero-mean, unit-variance 的 a_k 值。跨 group 拼接后的 batch-level std 仍为 ≈ 1 (除非 group 数量极少或 group 间 reward 分布高度非平稳)。

因此 Option A 的 Step 1.5 近似为恒等操作： $\hat{a}_k \approx a_k$ 。

Corollary 2 (两种方案的差异来源). Option A 与 Option B 的实质差异不在于“是否消除 per-reward scale 差异” (Step 1 的 group-std 已经做了)，而在于：

- Option A：聚合后不再 **normalize**。 $\mathcal{A}_{\text{final}}$ 的 variance 取决于 source s 的 OOD reward 数量：

$$\text{Var}(\mathcal{A}_{\text{final}}^A) = 1 + \frac{\gamma^2}{|\mathcal{R}_{\text{ood}}|} \quad (\text{假设各 } a_k \text{ 独立}) \quad (41)$$

对 geneval ($|\mathcal{R}_{\text{ood}}| = 1$): $\text{Var} = 1 + 0.04 = 1.04$

对 pickscore ($|\mathcal{R}_{\text{ood}}| = 0$): $\text{Var} = 1$

对 ocr ($|\mathcal{R}_{\text{ood}}| = 1$): $\text{Var} = 1.04$

$\Rightarrow$ 不同 source 的 final advantage 有微小的 variance 差异。

- Option B: σ_{global} 统一 rescale 所有 sample，确保 $\text{Var}(\mathcal{A}_{\text{final}}^B) \approx 1$ 对所有 sample。

12.3 对 NFT Loss 的影响

Proposition 6 (NFT Loss 对 Advantage Scale 的敏感性). 回顾 NFT loss 中 advantage 的作用 (eq. 4)：

$$r_s = \frac{\mathcal{A}_s + A_{\text{max}}}{2A_{\text{max}}} \in [0, 1] \quad (42)$$

其中 $A_{\text{max}} = \max(|\text{adv_clip_range}|)$ (配置中为 5.0)。

- A_{max} 起到 normalization 的作用：将 advantage 映射到 $[0, 1]$ 区间。
- 若 $\mathcal{A}_{\text{final}}$ 的实际 std $\ll A_{\text{max}}$ (如 $\text{std} \approx 1$ vs $A_{\text{max}} = 5$)，则 $r_s \approx 0.5 \pm 0.1$ ——advantage 的有效信号强度较弱。
- 若 $\mathcal{A}_{\text{final}}$ 的 std 变动较大，则不同 training epoch 的 r_s 分布不稳定。

结论：NFT loss 对 advantage 的 absolute scale 敏感度较低 (通过 A_{max} 缓冲)，但对不同 sample 间的相对大小高度敏感。因此 Option A 和 B 在 NFT loss 下的行为差异很小 ($\gamma = 0.2$ 导致的 variance 差异仅 4%)。

12.4 对比总结

	Option A (Before)	Option B (After)
Per-reward scale 消除	✓ 由 Step 1 group-std 完成 (Step 1.5 近似恒等)	✓ 同样由 Step 1 完成
Final advantage variance	依赖于 source 的 OOD reward 数量	统一 ≈ 1
γ 语义	精确: OOD 贡献 γ 比例的信号	精确 (同上, σ_{global} 只做 global rescale)
对 NFT loss 的稳定性	轻微不一致 (variance 差 $\sim 4\%$)	更稳定 (统一 scale)
鲁棒性	group-std 不精确时可能残留 scale 差异	σ_{global} 提供额外保险
计算开销	需要 per-reward 的 batch-std (K 次 std)	需要 combined 的 batch-std (1 次 std)
标准实践	非标准 (GRPO/GDPO 不使用此模式)	GDPO 的标准做法

12.5 推荐方案与理论依据

推荐: Option B (After Aggregation)

推荐 **batch-level normalization** 放在 **advantage** 聚合之后, 理由如下:

1. **Scale** 消除已由 **Step 1** 完成: Per-reward group-std 已将各 reward 归一化到 $\sim N(0, 1)$ 。Pre-aggregation batch-std 近似为恒等操作 (Proposition 5), 增加计算但无实质收益。
2. **Post-aggregation** σ_{global} 提供 **robustness**: 当 group size 较小 (如 $K = 4$) 时, per-group std 估计可能不精确, 导致聚合后的 advantage 的 batch-level std 偏离 1。 σ_{global} 除法提供了一个 “safety net”, 确保最终 advantage 的 scale 稳定。
3. 统一 **scale** 对 **NFT loss** 友好: 虽然 NFT loss 对 absolute scale 不太敏感 (有 A_{max} 缓冲), 但统一的 advantage scale 使 learning rate 的选择更稳定——不需要根据 OOD reward 数量调整 lr。
4. 与 **GRPO/GDPO** 一致: 标准 GRPO 使用 “group-mean $\rightarrow$ global-std” 的两层结构, MoF 的 “per-reward group-norm $\rightarrow$ aggregate $\rightarrow$ global-std” 是其自然扩展。

Remark 11 (何时 Option A 可能更好). 若未来引入 reward 间的非独立性 (如 geneval reward 与 pickscore reward 高度相关), 则 $\text{Var}(\mathcal{A}_{\text{combined}})$ 可能显著偏离 $1 + \gamma^2/|\mathcal{R}_{\text{ood}}|$ 的独立假设。此时 pre-aggregation normalization 可以消除相关性导致的 variance inflation。但在当前配置下 (rewards 相对独立), Option B 更简洁有效。

12.6 最终 Pipeline 伪代码

Algorithm 3 MoF Advantage Pipeline (Recommended: Option B)

Require: Raw rewards $\{R_k(x_i)\}_{k,i}$, group indices $\{g_i\}$, source labels $\{s_i\}$, gamma γ

Ensure: Per-sample advantage $\{\mathcal{A}_i\}$

// Step 1: Per-Reward Group Normalization

```

1: for each reward  $k$  do
2:   for each group  $g$  do
3:      $\mu_{k,g} \leftarrow \text{mean}(\{R_k(x_j) : g_j = g, R_k(x_j) \neq \text{NaN}\})$ 
4:      $\sigma_{k,g} \leftarrow \text{std}(\{R_k(x_j) : g_j = g, R_k(x_j) \neq \text{NaN}\})$ 
5:     for  $x_j$  in group  $g$  where  $R_k(x_j) \neq \text{NaN}$  do
6:        $a_k(x_j) \leftarrow (R_k(x_j) - \mu_{k,g}) / \max(\sigma_{k,g}, \epsilon)$ 
7:     end for
8:   end for
9: end for

```

// Step 2: Per-Set Advantage Aggregation

```

10: for each sample  $x_i$  from source  $s$  do
11:    $\text{in} \leftarrow$  in-domain reward for source  $s$ 
12:    $\mathcal{R}_{\text{ood}} \leftarrow$  applicable non-in-domain rewards for  $x_i$  (non-NaN)
13:    $\mathcal{A}_{\text{combined}}(x_i) \leftarrow a_{\text{in}}(x_i) + \gamma \cdot \text{mean}(\{a_j(x_i)\}_{j \in \mathcal{R}_{\text{ood}}})$ 
14: end for

```

// Step 3: Global-Std Normalization

```

15:  $\sigma_{\text{global}} \leftarrow \text{std}(\{\mathcal{A}_{\text{combined}}(x_j)\}_{j=1}^N)$ 
16: for each sample  $x_i$  do
17:    $\mathcal{A}_{\text{final}}(x_i) \leftarrow \mathcal{A}_{\text{combined}}(x_i) / \max(\sigma_{\text{global}}, \epsilon)$ 
18:    $\mathcal{A}_{\text{final}}(x_i) \leftarrow \text{clip}(\mathcal{A}_{\text{final}}(x_i), \text{adv\_clip\_min}, \text{adv\_clip\_max})$ 
19: end for

```

13 梯度消失分析与解决方案

13.1 问题现象

实验观察到 MoF 训练中 $\text{grad_norm} \sim 10^{-10}$, 且:

- 修改 $\text{nft_beta} = 1.0, 0.1, 0.01$ 只改变 loss 的 scale, 不改变 grad_norm 的数量级。
- Teacher-biased 初始化 ($C = 2$) 和 zeros 初始化表现相同数量级。
- EMA 与 current logits 的关系对 grad_norm 无显著影响。

13.2 根因分析: 逐步推导

我们从 loss $\mathcal{L}$ 到 logits $\ell_{j,i,s}$ 的完整梯度链出发, 逐步展示每个环节如何引入梯度压缩。

13.2.1 Step 1: 明确计算图

MoF 的前向计算图 (单 timestep i , 单 set s , 省略下标 i, s):

$$\underbrace{\ell \in \mathbb{R}^K}_{\text{logits}} \xrightarrow{\text{softmax}} \underbrace{\lambda \in \Delta^K}_{\text{weights}} \xrightarrow{\text{linear mix}} \underbrace{\sum_k \lambda_k v^{(k)} \in \mathbb{R}^d}_{\text{combined velocity}} \xrightarrow{\text{NFT loss}} \mathcal{L} \in \mathbb{R} \quad (43)$$

其中 $v^{(k)} \in \mathbb{R}^d$ 是 teacher k 的 **detached** (常数) velocity, $d = C \times H \times W = 65536$ 。

13.2.2 Step 2: Loss $\rightarrow$ Combined Velocity 的梯度 $\mathbf{g}$

定义 loss 对 combined velocity 的梯度向量:

$$\mathbf{g} \triangleq \frac{\partial \mathcal{L}}{\partial v^{\text{new}}} \in \mathbb{R}^d \quad (44)$$

这是一个 d -维向量, 由 NFT 的 MSE loss 决定。其具体形式为 (展开 NFT loss):
NFT loss 对 v^{new} 的梯度来自:

$$\mathcal{L} = \frac{1}{\beta} [r \cdot L^+(v^+) + (1-r) \cdot L^-(v^-)] \quad (45)$$

$$v^+ = \beta v^{\text{new}} + (1-\beta) v^{\text{old}} \Rightarrow \frac{\partial v^+}{\partial v^{\text{new}}} = \beta I_d \quad (46)$$

$$v^- = (1+\beta) v^{\text{old}} - \beta v^{\text{new}} \Rightarrow \frac{\partial v^-}{\partial v^{\text{new}}} = -\beta I_d \quad (47)$$

其中 v^{old} 是 detached 的 EMA combined velocity。代入链式法则:

$$\begin{aligned} \mathbf{g} = \frac{\partial \mathcal{L}}{\partial v^{\text{new}}} &= \frac{1}{\beta} \left[r \cdot \frac{\partial L^+}{\partial v^+} \cdot \beta + (1-r) \cdot \frac{\partial L^-}{\partial v^-} \cdot (-\beta) \right] \\ &= r \cdot \frac{\partial L^+}{\partial v^+} - (1-r) \cdot \frac{\partial L^-}{\partial v^-} \end{aligned} \quad (48)$$

关键发现 1: β 消去

β 在梯度中完全消去! 这数学性地解释了为什么修改 `nft_beta = 1.0, 0.1, 0.01` 不影响 `grad_norm`——只影响 loss 的绝对值 (scaling), 不影响 $\partial \mathcal{L} / \partial v^{\text{new}}$ 的方向或大小。

$\mathbf{g}$ 的每个分量 $\mathbf{g}[c, h, w]$ 由 NFT 的 self-normalized MSE 给出:

$$\mathbf{g}[c, h, w] = r \cdot \frac{2(x_0^+ - x_0)[c, h, w] \cdot (-\sigma_i)}{(w^+)^2 \cdot d} - (1-r) \cdot \frac{2(x_0^- - x_0)[c, h, w] \cdot \sigma_i}{(w^-)^2 \cdot d} \quad (49)$$

量级: 每个分量 $|\mathbf{g}[c, h, w]| \sim \frac{2\sigma_i \cdot |2r-1|}{d} \sim \frac{2 \times 0.45 \times 0.2}{65536} \approx 2.7 \times 10^{-6}$ 。

向量范数: $\|\mathbf{g}\| = \sqrt{\sum_{c,h,w} \mathbf{g}[c, h, w]^2} \sim \sqrt{d} \cdot 2.7 \times 10^{-6} \approx 6.9 \times 10^{-4}$ 。

13.2.3 Step 3: Combined Velocity $\rightarrow$ Weights 的梯度

由 $v^{\text{new}} = \sum_k \lambda_k v^{(k)}$ ($v^{(k)}$ 是常数), loss 对 weight λ_k 的梯度为标量:

$$\frac{\partial \mathcal{L}}{\partial \lambda_k} = \mathbf{g}^\top v^{(k)} = \sum_{c,h,w} \mathbf{g}[c, h, w] \cdot v^{(k)}[c, h, w] \quad (50)$$

这是两个 d -维向量的内积 (dot product), 结果是标量。

量级: $\mathbf{g}$ 的每个分量 $\sim 10^{-6}$, $v^{(k)}$ 的每个分量 $\sim O(1)$ 。若两者存在非零相关性 (alignment $\sim 30\%$), 则:

$$\left| \frac{\partial \mathcal{L}}{\partial \lambda_k} \right| \approx d \cdot 2.7 \times 10^{-6} \cdot 1.0 \cdot 0.3 \approx 0.05 \quad (51)$$

注意: 此时梯度量级完全合理 (~ 0.05)。

13.2.4 Step 4: Weights $\rightarrow$ Logits 的梯度 (Softmax Jacobian —灾难发生处)

Softmax 的 Jacobian 矩阵为:

$$\frac{\partial \lambda_k}{\partial \ell_j} = \frac{1}{\tau} \lambda_k (\delta_{kj} - \lambda_j) \quad (52)$$

其中:

- $\lambda_k = \text{softmax}(\ell/\tau)_k$ 是 teacher k 的当前 mixing weight (softmax 输出的第 k 个分量)
- ℓ_j 是 teacher j 对应的 logit 参数 (我们要求梯度的目标参数)
- δ_{kj} 是 Kronecker delta: $\delta_{kj} = 1$ if $k = j$, else 0
- τ 是 softmax temperature

通过链式法则, loss 对 logit ℓ_j 的梯度为:

$$\frac{\partial \mathcal{L}}{\partial \ell_j} = \sum_{k=1}^K \frac{\partial \mathcal{L}}{\partial \lambda_k} \cdot \frac{\partial \lambda_k}{\partial \ell_j} \quad (53)$$

$$= \sum_k \underbrace{\left(\mathbf{g}^\top v^{(k)} \right)}_{\text{Step 3 的结果}} \cdot \underbrace{\frac{\lambda_k (\delta_{kj} - \lambda_j)}{\tau}}_{\text{softmax Jacobian}} \quad (54)$$

其中 $\mathbf{g}^\top v^{(k)}$ 是 Step 3 中得到的 loss 对 λ_k 的梯度 (标量), 表示 “如果增加 teacher k 的权重, loss 会如何变化”。

展开求和 (将 $k = j$ 和 $k \neq j$ 的项分别处理):

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \ell_j} &= \frac{1}{\tau} \sum_k \left(\mathbf{g}^\top v^{(k)} \right) \lambda_k (\delta_{kj} - \lambda_j) \\ &= \frac{1}{\tau} \left[\underbrace{\left(\mathbf{g}^\top v^{(j)} \right) \lambda_j (1 - \lambda_j)}_{k=j \text{ 的贡献}} + \underbrace{\sum_{k \neq j} \left(\mathbf{g}^\top v^{(k)} \right) \lambda_k (0 - \lambda_j)}_{k \neq j \text{ 的贡献}} \right] \\ &= \frac{\lambda_j}{\tau} \left[\left(\mathbf{g}^\top v^{(j)} \right) (1 - \lambda_j) - \sum_{k \neq j} \lambda_k \left(\mathbf{g}^\top v^{(k)} \right) \right] \\ &= \frac{\lambda_j}{\tau} \left[\mathbf{g}^\top v^{(j)} - \lambda_j \mathbf{g}^\top v^{(j)} - \sum_{k \neq j} \lambda_k \mathbf{g}^\top v^{(k)} \right] \\ &= \frac{\lambda_j}{\tau} \left[\mathbf{g}^\top v^{(j)} - \sum_{k=1}^K \lambda_k \mathbf{g}^\top v^{(k)} \right] \end{aligned} \quad (55)$$

其中最后一步利用了 $\lambda_j \mathbf{g}^\top v^{(j)} + \sum_{k \neq j} \lambda_k \mathbf{g}^\top v^{(k)} = \sum_k \lambda_k \mathbf{g}^\top v^{(k)}$ 。

定义加权均值 velocity $\bar{v} = \sum_k \lambda_k v^{(k)}$ (即 combined velocity v^{new} 本身), 注意到:

$$\sum_k \lambda_k \mathbf{g}^\top v^{(k)} = \mathbf{g}^\top \left(\sum_k \lambda_k v^{(k)} \right) = \mathbf{g}^\top \bar{v} \quad (56)$$

代入得最终结果:

$$\boxed{\frac{\partial \mathcal{L}}{\partial \ell_j} = \frac{\lambda_j}{\tau} \mathbf{g}^\top (v^{(j)} - \bar{v})} \quad (57)$$

其中:

- ℓ_j : teacher j 的 logit 参数 (可学习标量)
- λ_j : teacher j 当前的 softmax 权重 ($\in [0, 1]$, 由 ℓ 决定)
- τ : softmax temperature (超参数)
- $\mathbf{g} = \partial \mathcal{L} / \partial v^{\text{new}} \in \mathbb{R}^d$: loss 对 combined velocity 的梯度向量 (由 NFT loss 的 MSE 决定)
- $v^{(j)} \in \mathbb{R}^d$: teacher j 的 velocity 输出 (frozen, detached 常数向量)
- $\bar{v} = \sum_k \lambda_k v^{(k)} \in \mathbb{R}^d$: 所有 teacher 的加权均值 velocity (= combined velocity v^{new})
- $v^{(j)} - \bar{v} \in \mathbb{R}^d$: teacher j 偏离当前混合均值的“差异方向”
- $\mathbf{g}^\top (v^{(j)} - \bar{v})$: d -维向量的内积 (标量), 衡量 loss gradient 方向与 teacher 差异方向的对齐程度

关键发现 2: Softmax 的“均值减除”效应

比较 Step 3 和 Step 4:

$$\text{对 } \lambda_j \text{ 的梯度: } \frac{\partial \mathcal{L}}{\partial \lambda_j} = \mathbf{g}^\top v^{(j)} \quad (\text{量级} \sim 0.05) \quad (58)$$

$$\text{对 } \ell_j \text{ 的梯度: } \frac{\partial \mathcal{L}}{\partial \ell_j} = \frac{\lambda_j}{\tau} \mathbf{g}^\top \underbrace{(v^{(j)} - \bar{v})}_{!!!} \quad (\text{量级} \sim 10^{-9}) \quad (59)$$

Softmax Jacobian 将“ $\mathbf{g}$ 对 $v^{(j)}$ 的 dot product”替换为“ $\mathbf{g}$ 对 $(v^{(j)} - \bar{v})$ 的 dot product”。直觉理解: Softmax 的归一化约束 ($\sum_k \lambda_k = 1$) 意味着增大 ℓ_j 必须同时减小其他 ℓ_k 的有效权重。梯度必须衡量“teacher j 比当前混合好多少”, 而非“teacher j 绝对好不好”。这个相对比较就是 $(v^{(j)} - \bar{v})$ 项的来源。

当所有 teacher 都来自同一 base model 时: $v^{(j)} \approx \bar{v} \approx v_{\text{base}}$ (因为 LoRA perturbation 很小), 所以 $(v^{(j)} - \bar{v})$ 的量级仅为 full velocity 的 1-5%。这就是灾难所在。

13.2.5 Step 5: 代入 LoRA 结构量化

对 LoRA teacher: $v^{(k)} = v_{\text{base}} + \delta^{(k)}$, 因此:

$$v^{(j)} - \bar{v} = (v_{\text{base}} + \delta^{(j)}) - (v_{\text{base}} + \bar{\delta}) = \delta^{(j)} - \bar{\delta} \quad (60)$$

Base model velocity 完全消去——logit 梯度仅由 LoRA 残差的差异驱动。

对于 $K = 2$ 均匀权重 ($\lambda_0 = \lambda_1 = 0.5$):

$$v^{(0)} - \bar{v} = \frac{1}{2}(v^{(0)} - v^{(1)}) = \frac{1}{2}(\delta^{(0)} - \delta^{(1)}) \quad (61)$$

最终:

$$\frac{\partial \mathcal{L}}{\partial \ell_0} = \frac{0.5}{\tau} \mathbf{g}^\top \frac{\delta^{(0)} - \delta^{(1)}}{2} = \frac{1}{4\tau} \mathbf{g}^\top (\delta^{(0)} - \delta^{(1)}) \quad (62)$$

量级: $|\mathbf{g}^\top (\delta^{(0)} - \delta^{(1)})| \sim \|\mathbf{g}\| \cdot \|\delta^{(0)} - \delta^{(1)}\| \cdot |\cos \theta|$, 其中:

- $\|\mathbf{g}\| \sim 6.9 \times 10^{-4}$
- $\|\delta^{(0)} - \delta^{(1)}\| \sim \sqrt{d} \cdot 0.02 \approx 5.1$ (LoRA diff per element ~ 0.02)
- $|\cos \theta| \sim 1/\sqrt{d} \approx 0.004$ ($\mathbf{g}$ 和 δ -diff 方向无先验相关性)

$$\left| \frac{\partial \mathcal{L}}{\partial \ell_0} \right| \sim \frac{1}{4} \times 6.9 \times 10^{-4} \times 5.1 \times 0.004 \approx 3.5 \times 10^{-6} \quad (63)$$

再除以 $\text{GAS} = 80: \approx 4.4 \times 10^{-8}$ 。对 $K \times T \times S = 40$ 个参数的 grad norm: $\sqrt{40} \times 4.4 \times 10^{-8} \approx 2.8 \times 10^{-7}$ 。

考虑到 $|\cos \theta|$ 的实际值可能更小 (LoRA diff 和 MSE gradient 几乎正交), 进一步压缩 1-2 个数量级到 $\sim 10^{-9}$ - 10^{-10} 。

13.3 为什么 `nft_beta` 不影响梯度

如 Step 2 公式 (48) 所示, $\mathbf{g} = r \cdot \partial L^+ / \partial v^+ - (1-r) \cdot \partial L^- / \partial v^-$, 与 β 完全无关。 β 只出现在 v^+, v^- 的定义中和 loss 的 $1/\beta$ 前缀中, 两者的贡献在链式法则中精确消去。

13.4 梯度消失的三重因素总结

梯度消失的三重因素

$$\underbrace{|\nabla_{\ell} \mathcal{L}|}_{\sim 10^{-10}} = \underbrace{\frac{\lambda_j}{\tau} \cdot \|\mathbf{g}\|}_{\text{NFT loss gradient}} \times \underbrace{|\cos \theta| \cdot \|\delta^{(j)} - \bar{\delta}\|}_{\text{Softmax direction projection}} \times \underbrace{\frac{1}{\text{GAS} \cdot \text{DDP}}}_{\text{Distributed scaling}} \quad (64)$$

1. Softmax Mean Subtraction (主因, $\sim 100\times$ reduction):

Softmax 的 Jacobian 使梯度正比于 $v^{(j)} - \bar{v}$ ——teacher j 偏离加权均值的方向。对于共享 base model 的 LoRA teacher:

$$\|v^{(j)} - \bar{v}\| = \|\delta^{(j)} - \bar{\delta}\| \ll \|v^{(j)}\| \quad (65)$$

LoRA perturbation 仅占 base velocity 的 1-5%。任何 **normalized mixing (softmax, sigmoid for $K=2$, 或任何满足 $\sum w_k = 1$ 的参数化)** 都有此性质。

2. High-Dimensional Dot Product Cancellation (~ 10 - $100\times$ reduction):

梯度是 $\mathbf{g} \in \mathbb{R}^d$ 与 $(v^{(j)} - \bar{v}) \in \mathbb{R}^d$ 的 dot product。两者方向无先验相关性 (loss gradient 指向 “prediction error 方向”, 而 teacher difference 指向 “LoRA 差异方向”), 在 $d = 65536$ 维空间中:

$$\mathbb{E}[|\cos \theta|] \sim \frac{1}{\sqrt{d}} \approx 0.004 \quad (66)$$

3. GAS + DDP Division ($\sim 640\times$ reduction):

Gradient accumulation steps = $\frac{\text{num_batches}}{\text{gradient_step_per_epoch}} \times T = 8 \times 10 = 80$, 加上 DDP 8-GPU all-reduce averaging。

总压缩比: $\sim 0.02 \times 0.004 \times (1/80) \approx 10^{-6}$, 将 $\sim 10^{-4}$ 量级的 “理想梯度” 压缩至 $\sim 10^{-10}$ 。

13.5 核心困难的数学本质

Theorem 5 (Normalized Mixing 的梯度瓶颈). 对任何满足 $\sum_k w_k = 1$ 的参数化 $w_k = f_k(\theta)$ (包括 softmax, sigmoid for $K=2$, Gumbel-softmax 等), combined velocity $v_\theta = \sum_k w_k v^{(k)}$ 的梯度满足:

$$\frac{\partial v_\theta}{\partial \theta} = \sum_k \frac{\partial w_k}{\partial \theta} v^{(k)} = \sum_k \frac{\partial w_k}{\partial \theta} (v^{(k)} - v_\theta + v_\theta) \quad (67)$$

由约束 $\sum_k w_k = 1 \Rightarrow \sum_k \frac{\partial w_k}{\partial \theta} = 0$ ，因此：

$$\boxed{\frac{\partial v_\theta}{\partial \theta} = \sum_k \frac{\partial w_k}{\partial \theta} (v^{(k)} - v_\theta)} \quad (68)$$

梯度恒等式：无论参数化方式如何，normalized mixing 的梯度必定正比于 teacher velocities 与其加权均值的偏差。当 teachers 共享 base model 时，此偏差本质上是 LoRA residual 差异——量级远小于 velocity 本身。

这是数学约束，非 **softmax** 特有。

13.6 对比：为什么 LLM 的 Logits + Softmax 没有此问题

对比：LLM Head (Softmax) vs MoF (Softmax)

LLM 的设置：

$$p_i = \text{softmax}(Wh)_i, \quad \text{其中 } W \in \mathbb{R}^{V \times d_{\text{model}}}, h \in \mathbb{R}^{d_{\text{model}}} \quad (69)$$

Loss: $\mathcal{L} = -\log p_y$ (cross-entropy with target y)。

梯度对 logits $z = Wh$ ：

$$\frac{\partial \mathcal{L}}{\partial z_j} = p_j - \mathbf{1}[j = y] \quad (70)$$

梯度对 weight W ：

$$\frac{\partial \mathcal{L}}{\partial W_{j,:}} = (p_j - \mathbf{1}[j = y]) \cdot h^\top \quad (71)$$

为什么 LLM 不梯度消失？关键差异：

1. LLM 的 softmax “inputs” 是可学习的 logits $z = Wh$ (高维参数矩阵的输出)，不是 mixing weights 作用于 fixed 高维向量。

MoF 的计算图： $\ell \xrightarrow{\text{softmax}} \lambda \xrightarrow{\text{dot with } v^{(k)} \in \mathbb{R}^{65536}} \text{scalar outputs for loss}$

LLM 的计算图： $W, h \xrightarrow{\text{linear}} z \xrightarrow{\text{softmax}} p \xrightarrow{\text{CE loss}} \text{scalar}$

2. LLM 中 softmax 的“选项” (V 个 vocabulary tokens) 彼此完全不同。

对 LLM, z_j 和 z_k (不同 token 的 logit) 是独立参数的独立输出，没有“共享 base”的结构。梯度 $p_j - \mathbf{1}[j = y]$ 的量级 $\sim O(1)$ (如 $p_y \approx 0.9$ 时梯度 $= -0.1$)。

对 MoF, $v^{(0)}$ 和 $v^{(1)}$ 共享 99% 的内容 (同一 base model + 小 LoRA)。Softmax 的 mean-subtraction 消去了共享部分，只留下 1% 的 LoRA 差异。

3. 维度角色不同：

LLM 中 softmax 作用于 输出空间 (V 个 token 概率)，梯度直接是 $O(1)$ 标量信号 ($p_j - \mathbf{1}[j = y]$)，不涉及高维 dot product。

MoF 中 softmax 作用于 K 个 teacher 的权重 (仅 2-3 个)，然后乘以 $d = 65536$ 维的 teacher velocity，梯度需要通过 $\mathbb{R}^d$ 空间的 dot product 传递——方向对齐性 $\sim 1/\sqrt{d}$ 导致信号大量 cancellation。

4. 总结：

	LLM (lm_head + softmax)	MoF (logits + softmax + velocity mix)
Softmax 选项数	$V \sim 32000-128000$	$K = 2-3$
选项间差异	z_j, z_k 完全独立 (无共享结构)	$v^{(j)}, v^{(k)}$ 共享 95-99% (base model)
梯度 bottleneck	无 ($\partial \mathcal{L} / \partial z_j = p_j - \mathbf{1}[j = y] \sim O(1)$)	$\mathbf{g}^\top (v^{(j)} - \bar{v}) \sim O(10^{-6})$ (高维 dot product + LoRA diff)
高维 dot product	无 (loss 直接在概率空间定义)	有 ($\mathbb{R}^{65536}$ 维 velocity dot product)
梯度量级	$\sim 0.01-1.0$	$\sim 10^{-10}$

根本区别：LLM 的 loss (cross-entropy) 直接在 softmax 输出概率空间上定义，梯度 $= p_j - \text{target}$ 是 $O(1)$ 标量。MoF 的 loss (NFT MSE) 定义在 softmax 输出之后的高维 velocity 空间上，梯度需要从 $\mathbb{R}^d$ 投影回 $\mathbb{R}^K$ 的 logit 空间——这个投影 (dot product + mean subtraction) 造成了灾难性的信号衰减。

14 解决方案分析

14.1 Solution A: Direct Reward Loss (Contextual Bandit)

14.1.1 动机

NFT loss 的梯度需经过高维 velocity 空间的 dot product, 被 teacher 相似性压缩。直接在 logit 空间用 reward 信号更新权重, 完全绕过 velocity dot product。

14.1.2 Loss 定义

Definition 8 (Reward-Weighted Log-Likelihood Loss). 对 batch 中 prompt set s 的 sample x_b , 设 per-sample advantage 为 $\mathcal{A}_b$:

$$\mathcal{L}_s^{\text{bandit}} = -\frac{1}{B_s \cdot T} \sum_{b \in \mathcal{B}_s} \sum_{i=1}^T \mathcal{A}_b \cdot \log \lambda_{\hat{k}(b,i), i, s} \quad (72)$$

其中 $\hat{k}(b, i)$ 是样本 b 在 timestep i 上的 “oracle best teacher”:

$$\hat{k}(b, i) = \arg \max_k \mathbf{g}_{i,s}^{(b)} \cdot \mathbf{v}_i^{(k)} \quad (\text{NFT gradient 方向上最优的 teacher}) \quad (73)$$

Soft 版本 (无需 argmax, 更平滑):

$$\mathcal{L}_s^{\text{soft}} = -\frac{1}{B_s \cdot T} \sum_{b \in \mathcal{B}_s} \sum_{i=1}^T \sum_k \alpha_{k,i}^{(b)} \mathcal{A}_b \cdot \log \lambda_{k,i,s} \quad (74)$$

其中 $\alpha_{k,i}^{(b)}$ 是 teacher k 在 sample b , timestep i 上的 “contribution indicator” (如 softmax of per-teacher rewards)。

14.1.3 梯度分析

Proposition 7 (Bandit Loss Gradient). 对 soft bandit loss (74), 梯度为:

$$\frac{\partial \mathcal{L}_s^{\text{soft}}}{\partial \ell_{j,i,s}} = -\frac{1}{\tau B_s} \sum_{b \in \mathcal{B}_s} \mathcal{A}_b \left[\alpha_{j,i}^{(b)} - \lambda_{j,i,s} \sum_k \alpha_{k,i}^{(b)} \right] \quad (75)$$

当 $\sum_k \alpha_{k,i}^{(b)} = 1$ (probability assignment) 时简化为:

$$\frac{\partial \mathcal{L}_s^{\text{soft}}}{\partial \ell_{j,i,s}} = -\frac{1}{\tau B_s} \sum_b \mathcal{A}_b \left(\alpha_{j,i}^{(b)} - \lambda_{j,i,s} \right) \quad (76)$$

梯度量级: $\sim \frac{|\mathcal{A}|}{B_s \cdot \tau} \cdot |\alpha_j - \lambda_j| \sim 0.01\text{--}0.1$ 。

关键优势

Bandit gradient (76) 依赖于:

- $\mathcal{A}_b$: advantage 量级 $\sim 0.2-2.0$
- $\alpha_j - \lambda_j$: 当前权重与“目标权重”的差异 $\sim 0.1-0.5$

完全不依赖 **teacher velocity** 差异 $\|v^{(j)} - \bar{v}\|$ 。

对比 NFT gradient (??):

	NFT Gradient	Bandit Gradient
梯度量级	$\sim 10^{-10}$	$\sim 10^{-2}$
依赖 teacher 相似性	✓ (正比于 $\ \delta^{(j)} - \bar{\delta}\ $)	✗
依赖 high-dim dot product	✓ ($\cos \theta \sim 1/\sqrt{d}$)	✗
受 β 影响	✗ (cancelled)	N/A

14.1.4 实际实现: Per-Teacher Advantage

需要为每个 teacher 获得独立的 advantage 信号。两种方案:

方案 **A1: Counterfactual per-teacher reward.** 对每个 sample b , 在采样阶段记录每个 teacher 的“贡献度”:

$$\alpha_{k,i}^{(b)} = \frac{\exp(\text{sim}(v_i^{(k)}, v_i^*))}{\sum_{k'} \exp(\text{sim}(v_i^{(k')}, v_i^*))} \quad (77)$$

其中 $v^* = (x_t - x_0)/\sigma$ 是 ground truth velocity。 α 衡量每个 teacher 在该 timestep 上与 ground truth 的对齐程度。

方案 **A2: Per-teacher baseline evaluation.** 在每个 eval epoch 中对每个 teacher 单独生成图像并计算 reward:

$$\alpha_{k,i}^{(b)} = \begin{cases} 1 & \text{if } k = \arg \max_{k'} R_{\text{in}(s)}(\text{img from teacher } k') \\ 0 & \text{otherwise} \end{cases} \quad (78)$$

14.2 Solution B: REINFORCE on Logit Space

14.2.1 Loss 定义

Definition 9 (REINFORCE Policy Gradient for Mixing Weights). 将 $\lambda_{k,i,s}$ 视为在每个 timestep 上“选择 teacher k ”的概率, 使用 policy gradient:

$$\nabla_{\ell_{:,i,s}} J_s = \mathbb{E}_{k \sim \lambda_{:,i,s}} [\mathcal{A}_s \cdot \nabla_{\ell_{:,i,s}} \log \lambda_{k,i,s}] \quad (79)$$

Soft relaxation (不采样, 直接用期望):

$$\mathcal{L}_s^{\text{PG}} = -\frac{1}{T} \sum_{i=1}^T \mathcal{A}_s \cdot \sum_k \lambda_{k,i,s} \log \lambda_{k,i,s} \quad (80)$$

14.2.2 梯度分析

Proposition 8 (REINFORCE Gradient). 对 soft PG loss (80):

$$\frac{\partial \mathcal{L}_s^{\text{PG}}}{\partial \ell_{j,i,s}} = -\frac{\mathcal{A}_s}{\tau} \sum_k \lambda_{k,i,s} (\delta_{kj} - \lambda_{j,i,s}) (\log \lambda_{k,i,s} + 1) \quad (81)$$

简化 (对 $K = 2$, uniform $\lambda = 0.5$):

$$\left| \frac{\partial \mathcal{L}_s^{\text{PG}}}{\partial \ell_{j,i,s}} \right| \sim \frac{|\mathcal{A}_s|}{4\tau} \cdot |\log(0.5) + 1| = \frac{|\mathcal{A}_s|}{4\tau} \cdot 0.307 \quad (82)$$

量级: $\sim |\mathcal{A}_s|/13 \sim 0.02\text{--}0.15$ 。同样不依赖 teacher velocity 差异。

Remark 12 (REINFORCE 的局限性). Soft PG loss (80) 本质上最大化 entropy $H(\lambda_{:,i,s})$ weighted by advantage。它推动权重向 high-advantage teacher 集中, 但:

- 只有一个 advantage $\mathcal{A}_s$ per sample (不是 per-teacher advantage)。
- 所有 timestep 共享同一 $\mathcal{A}_s \rightarrow$ 无法区分哪个 timestep 的哪个 teacher 贡献了高 reward。
- 需要 per-teacher counterfactual reward 才能提供 per- (k, i) 的 gradient signal。

14.3 Solution C: Residual-Space NFT

14.3.1 动机

保持 NFT 框架, 但在数值上放大 teacher 差异信号。将 velocity 分解为 base + residual, 在 residual 空间操作。

14.3.2 Loss 定义

Definition 10 (Residual-Space NFT). 显式计算 base velocity 和 per-teacher residuals:

$$v_{\text{base}} = \frac{1}{K} \sum_k v^{(k)} \quad (\text{teacher 均值作为 base, detached}) \quad (83)$$

$$\delta^{(k)} = v^{(k)} - v_{\text{base}} \quad (\text{per-teacher residual}) \quad (84)$$

Combined velocity 重写为:

$$v_{i,s}^{\text{new}} = v_{\text{base}} + \sum_k \lambda_{k,i,s} \delta_i^{(k)} \quad (85)$$

NFT loss 不变, 但 loss gradient $\mathbf{g}$ 现在需与 $\delta^{(k)} - \bar{\delta}$ 做 dot product。

14.3.3 梯度分析

Proposition 9 (Residual-Space 梯度). 数学上, residual decomposition 不改变梯度:

$$\frac{\partial \mathcal{L}}{\partial \ell_{j,i,s}} = \frac{\lambda_j}{\tau} \mathbf{g}^\top (\delta^{(j)} - \bar{\delta}) = \frac{\lambda_j}{\tau} \mathbf{g}^\top (v^{(j)} - \bar{v}) \quad (\text{恒等}) \quad (86)$$

因为 v_{base} 是常数, $\sum_k \lambda_k = 1$, base 项在 softmax Jacobian 中消去。

但数值实现上可以有改进: 若在 fp32 下显式计算 $\delta^{(k)} = v^{(k)} - v_{\text{base}}$ 并存储 (避免在 bf16 backward 中隐式相减), 可减少精度损失。

Remark 13 (Partial Fix 的量级改善). Residual-space 实现的主要收益:

- 避免 bf16 下隐式计算 $v^{(0)} - v^{(1)}$ (大数减大数的精度损失)
- 但数学上梯度量级不变——Theorem 5 的约束无法绕过
- 预期改善: $\sim 2\text{--}10\times$ (bf16 precision recovery), 不足以解决 $10^8\times$ 的差距

14.4 Solution D: Unnormalized Additive Mixing

14.4.1 动机

打破 Theorem 5 的约束 $\sum w_k = 1$ 。使用不归一化的 per-teacher scaling，使梯度正比于 $\delta^{(k)}$ 本身而非 $\delta^{(k)} - \bar{\delta}$ 。

14.4.2 Loss 定义

Definition 11 (Unnormalized Additive Velocity Mixing).

$$v_{i,s}^{\text{new}} = v_{\text{base}}(x, t_i) + \sum_{k=1}^K \alpha_{k,i,s} \delta^{(k)}(x, t_i) \quad (87)$$

其中 $\alpha_{k,i,s} = \ell_{k,i,s}$ (直接使用 logit, 无 softmax), $v_{\text{base}} = v^{(\text{ref})}$ 是参考模型 (base model 或 teacher 均值), $\delta^{(k)} = v^{(k)} - v_{\text{base}}$ 。

Loss 仍为 NFT loss, 但以 v^{new} 的新定义代入。

14.4.3 梯度分析

Proposition 10 (Unnormalized Gradient).

$$\frac{\partial \mathcal{L}}{\partial \ell_{j,i,s}} = \mathbf{g}_{i,s}^\top \delta_i^{(j)} \quad (88)$$

无 **mean subtraction**! 梯度直接为 loss gradient $\mathbf{g}$ 与 LoRA residual $\delta^{(j)}$ 的 dot product。量级估计:

$$\left| \frac{\partial \mathcal{L}}{\partial \ell_{j,i,s}} \right| \approx \|\mathbf{g}\| \cdot \|\delta^{(j)}\| \cdot |\cos \phi| \quad (89)$$

$$\sim 2.6 \times 10^{-4} \cdot 2.6 \cdot 0.004 \quad (90)$$

$$\sim 2.7 \times 10^{-6} \quad (\text{per timestep, before GAS}) \quad (91)$$

After GAS: $\sim 3.4 \times 10^{-8}$. Grad norm: $\sim 10^{-7}$.

改善: $\sim 10^3 \times$ vs NFT's 10^{-10} , 因为去掉了 $(\delta^{(j)} - \bar{\delta}) \rightarrow \delta^{(j)}$ 的 mean subtraction 因子。

Remark 14 (Unnormalized Mixing 的代价). • Combined velocity 不再是 teacher 的凸组合 $\rightarrow$ 可能超出 teacher velocity 的凸包

- 需要正则化约束 α_k 的范围 (如 L_2 penalty 或 clip)
- Level 1 保证 ($\geq$ single teacher) 不再自动成立——需额外约束 $\sum_k \alpha_k \approx 1$
- 实际收益有限 ($\sim 10^3 \times$ 改善仍不如 Solution A 的 $\sim 10^8 \times$)

14.5 Solution 对比总表

	NFT (current)	A: Direct Reward	B: REINFORCE	D: Unnormalized
Gradient formula	$\frac{\lambda_j}{\tau} \mathbf{g} \cdot (v^{(j)} - \bar{v})$	$-\frac{\mathcal{A}}{B\tau}(\alpha_j - \lambda_j)$	$-\frac{\mathcal{A}}{4\tau} \cdot 0.3$	$\mathbf{g} \cdot \delta^{(j)}$
Gradient magnitude	$\sim 10^{-10}$	$\sim 10^{-2}$	$\sim 10^{-2}$	$\sim 10^{-7}$
Depends on $\ v^{(j)} - \bar{v}\ $	✓	✗	✗	Partial
Depends on $\cos \theta$	✓	✗	✗	✓
Level-1 guarantee	✓	✓ (with init)	✓ (with init)	Needs constraint
Per-timestep signal	✓	✓ (with $\alpha_{k,i}$)	✗ (shared $\mathcal{A}$)	✓
改进倍数 vs current	$1\times$	$\sim 10^8\times$	$\sim 10^8\times$	$\sim 10^3\times$

14.6 推荐方案

推荐: Solution A (Direct Reward Loss)

核心思路: 不通过高维 velocity 空间传递梯度, 直接用 reward/advantage 信号在 logit 空间做梯度更新。

实现方式:

1. 保持 sampling/inference 不变 (仍用 softmax combined velocity 生成图像)
2. 计算 reward 和 advantage 不变
3. 替换 optimize 中的 NFT loss 为:

$$\mathcal{L} = -\frac{1}{B_s T} \sum_b \sum_i \mathcal{A}_b \cdot \log \lambda_{\hat{k}(b,i),i,s} \quad (92)$$

4. 其中 $\hat{k}(b,i)$ 可简化为 “in-domain teacher index” (teacher-biased REINFORCE), 或由 per-teacher reward evaluation 确定

优势:

- 梯度量级 $\sim 10^{-2}$ ($\sim 10^8 \times$ improvement)
- 完全不依赖 teacher velocity 相似性
- 与 NFT 框架正交——可作为 “warm start” 后切换到 NFT 的策略
- 保持 Level-1 guarantee (teacher-biased init + reward-driven update only increases in-domain)

潜在问题:

- Per-timestep teacher attribution ($\hat{k}(b,i)$ or $\alpha_{k,i}$) 需要额外计算
- 若只有 per-sample advantage (不是 per- (k,i) advantage), 所有 timestep 收到相同信号 $\rightarrow$ 学不到 timestep-specific 策略
- 可能需要与 NFT loss 混合使用 (NFT 提供 per-timestep direction, bandit 提供 magnitude)